MODULE 10 · DOMAIN 2 - ACCOUNT MANAGEMENT AND DATA GOVERNANCE

Masking, row access, tags, classification and lineage

The governance half of Domain 2. Where the Enterprise policy family from module 2 finally gets used.

DOMAIN 2 of 5 WEIGHT 20% MODULE 10 of 31

admin@sthithapragna.com

A policy is an **object**, created once and **attached** to many things. It is not a property of a column.

The policy family

POLICY	WHAT IT CONTROLS
Masking	Rewrites a column value at query time. The stored data is unchanged.
Row access	Which rows a query returns at all.
Projection	Whether a column may appear in the output. It can still be joined on.
Aggregation	Forces queries to aggregate over a minimum group size.
Join	Forces a table to be joined before it can be read.

ON THE EXAM

All five are schema-level objects and all five need Enterprise. Masking and row access are the two you will be asked about most.

Masking: the part people miss

- Applied **at query runtime**. The underlying data is never altered, and different roles see different results from the same query.
- **Applied wherever the column is referenced.** Not just the SELECT list: **JOIN predicates, WHERE, ORDER BY and GROUP BY**.
- That is deliberate. It prevents de-anonymisation by filtering or joining on a column you cannot see.
- Documented consequence: a query touching masked columns **may not give the anticipated result**, because the mask blocks proper evaluation.
- Work on **tables and views**. Use **POLICY_CONTEXT** to simulate a query as another role.

> ON THE EXAM
>
> 'The filter returns no rows even though I know they exist' is a masking symptom, not a data problem.

Conditional masking and row access policies

- **Conditional masking** lets the policy read *another column* to decide. Mask the email unless a consent flag is set.
- A **row access policy** is attached to a **table or view**, not to a column, and filters rows out entirely.
- It typically compares the current role, or a mapping table, against a column value on the row.
- Rows removed by a row access policy **are not counted** in aggregations over the table.

> ON THE EXAM
>
> Columns or rows. Masking changes what a value looks like; a row access policy decides whether the row is there at all.

The documented order of evaluation

01

Row access

Filter rows out. Everything downstream sees only what survives.

02

Projection

Check whether any restricted column is being projected. If so, reject the query.

03

Masking

Apply column masks to what remains.

ON THE EXAM

Rows first, then whether the column may be shown at all, then what the value looks like. Each step narrows what the next one sees.

Projection policy or masking policy?

MASKING POLICY

The column appears, changed

- Value is rewritten at query time
- The column is still in the result set
- Applied everywhere the column is referenced
- 'Show the last four digits only'

PROJECTION POLICY

The column cannot appear

- The query is rejected if the column is projected
- The column can still be joined on or grouped by
- One projection policy per column at a time
- 'Join on the hash, never display it'

A DETAIL WORTH KEEPING

Projection policies are ignored in nested queries. They govern only what reaches the final result.

Object tagging

- A **tag** is a key/value label attached to tables, columns, schemas, databases, warehouses and more.
- **Tag lineage:** child objects **inherit tags from their parent**. Tag a schema and every table and column beneath it carries the tag.
- **ACCOUNT_USAGE.TAGS** catalogues every tag in the account, current and deleted.
- **TAG_REFERENCES_ALL_COLUMNS** returns every tag on every column of a table or view.

> ON THE EXAM
>
> Tags are also how cost gets attributed. Tag a warehouse by cost centre and the credits become attributable.

Tag-based masking: the scaling answer

- Attach a masking policy **to a tag** rather than to a column.
- Every column carrying that tag is **automatically protected**. No per-column attachment.
- Combined with classification, **newly discovered sensitive columns are masked as the tag is applied**.
- Change the policy once and every tagged column changes with it.

> ON THE EXAM
>
> 'We have hundreds of tables with email columns and need consistent masking' has exactly one good answer, and this is it.

Sensitive data classification

- Every column found to be sensitive gets **two** categories, and the exam expects both names.
- **Semantic category:** the *type* of attribute. Name, email address, credit card number.
- **Privacy category:** the *sensitivity*. **IDENTIFIER, QUASI_IDENTIFIER, or SENSITIVE.**
- Applied as **system tags** in the SNOWFLAKE.CORE schema: SEMANTIC_CATEGORY and PRIVACY_CATEGORY.
- A **tag map** can translate a system tag into your own user-defined tag automatically.

> ON THE EXAM
>
> Semantic is what the data is. Privacy is how dangerous it is. A postcode is a QUASI_IDENTIFIER: not identifying alone, but identifying in combination.

Classification profiles, and the defaults

- A **classification profile** holds the settings. **auto_tag** decides whether tags are applied or merely recommended.
- Default behaviour: **reclassifies every 30 days**, scans all native semantic categories.
- **Views are excluded** from classification.
- Classification samples **up to 10,000 randomly selected rows per table**. It does not read everything.
- Cost appears in METERING_HISTORY under service type **SENSITIVE_DATA_CLASSIFICATION**.

> ON THE EXAM
>
> Sampling matters. A sensitive value appearing in a small fraction of rows may not be found. Classification assists judgement, it does not replace it.

Lineage, Trust Center and Horizon Catalog

- **Data lineage** in Snowsight traces how data flows between objects, upstream and downstream.
- **Trust Center** is the Snowsight surface for security posture and for setting up classification without writing SQL.
- **Horizon Catalog** is the umbrella name for the whole governance surface, not a separate product.
- Under it: classification, tagging, **data metric functions**, masking and row access policies, lineage, **Universal Search** and Trust Center.

> ON THE EXAM
>
> Several Horizon components pre-date the name. If a question describes tagging or masking as 'part of Horizon', that is just current branding.

Which control for which requirement

THE REQUIREMENT	THE CONTROL
Analysts see only the last four digits of a card number	Masking policy
European staff see only European customers' rows	Row access policy
A partner may join on an identifier but never display it	Projection policy
No query may return a group smaller than 25 people	Aggregation policy
The same rule must apply across hundreds of tables	Tag-based masking
Find the sensitive columns we do not know about	Classification
Show where this figure came from	Data lineage

ON THE EXAM

Seven rows covering the module. Rows 1 to 3 are the ones the exam most likes to blur together.

A masking policy on CUSTOMER.EMAIL returns a hashed value for the analyst role. An analyst reports that a query joining CUSTOMER to a marketing table on EMAIL now returns almost no rows, although it worked before the policy was applied.

They also report that WHERE EMAIL = 'known@example.com' returns nothing, despite that row certainly existing.

What is happening?

A The masking policy is misconfigured and is masking rows as well as values.

B A row access policy must also have been applied to the table at the same time.

C Masking is applied wherever the column is referenced, including JOIN and WHERE predicates.

D Masked columns cannot be used in joins at all, so the query should have raised an error.

◆ PAUSE AND DECIDE

What is happening?

A The policy is masking rows as well.

Masking policies act on column values only. Removing rows is what a row access policy does, and none was applied.

B A row access policy was also applied.

Would explain missing rows, and the scenario describes only a masking policy. It also would not explain the WHERE clause failing on a known value.

C Masking applies to JOIN and WHERE predicates too.

Documented behaviour. The join compares hashes to plain text, and the WHERE compares a hash to a literal. Neither matches.

D Masked columns cannot be joined.

They can be joined. The join runs and simply matches nothing, which is why there is no error to help the analyst.

THE SIGNAL

Deliberate design. If masking applied only to output, you could recover values by filtering for them one at a time.

A retailer has around 900 tables across twelve databases. Many contain a column holding a customer email address, under a variety of names. New tables appear weekly from several pipelines.

Compliance requires that every email column is masked identically for all roles except a small privacy team. That applies now, and to tables that do not yet exist.

What is the appropriate design?

A Classify the databases, apply a tag to the identified columns, and attach the masking policy to the tag.

B Write a masking policy and attach it to each email column, with a weekly job to catch new tables.

C Create a secure view over every table exposing a masked email, and revoke access to the base tables.

D Apply a row access policy at database level, which cascades to all tables including future ones.

◆ PAUSE AND DECIDE

What is the appropriate design?

A Classify, tag, and attach the policy to the tag.

Classification finds the columns whatever they are named, the tag map applies the tag, and tag-based masking attaches the policy automatically.

B Attach per column, plus a weekly job.

Works today and leaves a gap every week between a table appearing and the job running. It is also 900 attachments to maintain.

C Secure views over every table.

900 views to build and keep in step with schema changes, and analysts lose access to the base tables entirely.

D A row access policy at database level.

Wrong policy type. Row access filters rows, not column values. Policies are also attached to objects, not cascaded from a database.

THE SIGNAL

'Under a variety of names' rules out anything based on matching column names. That phrase is why classification is in the answer.

A bank must ensure that relationship managers can query the shared accounts table but see only the accounts belonging to their own branch. Branch membership is held in a separate mapping table that links each role to its branch codes.

Managers must be able to run SUM and COUNT over the table, and those aggregates must reflect only their own branch.

What should be implemented?

A. A masking policy on the branch code column, so other branches' values are hidden.

B. A projection policy on the branch code column, preventing other branches being displayed.

C. An aggregation policy, which restricts results to the querying role's own group.

D. A row access policy on the table, referencing the mapping table to filter rows.

◆ PAUSE AND DECIDE

What should be implemented?

A Mask the branch code column.

Masking hides a value; the rows remain. Managers would still see other branches' balances, with the branch obscured. Worse, not better.

B A projection policy on branch code.

Projection policies govern whether a column may be displayed. They do not vary by row and would block the column for everyone.

C An aggregation policy.

Aggregation policies enforce a minimum group size to prevent isolating individuals. They do not scope results to a role.

D A row access policy referencing the mapping table.

Filters rows by role using a mapping table, and excluded rows are not counted in aggregations. Both requirements met.

THE SIGNAL

'Aggregates must reflect only their own branch' is the confirming detail: rows excluded by a row access policy are not counted.

> A data provider shares a table with a partner. The partner must be able to join their own records to the shared table on a hashed customer identifier, and to group results by it. Under no circumstances may the hashed identifier appear in any result the partner sees.
>
> A masking policy already replaces the raw identifier with the hash for all external roles.

What should be added?

A. A second masking policy on the hash column, returning NULL for external roles.

B. A projection policy on the hash column, denying projection to the partner's role.

C. An aggregation policy, since the partner only needs grouped results.

D. Remove the column from the shared secure view, since joins do not require the column to be present.

◆ PAUSE AND DECIDE

What should be added?

A. A second masking policy returning NULL.
Masking applies everywhere the column is referenced, so the join would compare NULL to NULL and break the requirement to join.

B. A projection policy denying projection.
Exactly the purpose: the column cannot be projected into the result, but remains usable in JOIN and GROUP BY.

C. An aggregation policy.
Enforces a minimum group size. It says nothing about whether a particular column may be displayed.

D. Remove the column from the view.
A column absent from the view cannot be joined on either. It removes the capability the partner needs.

THE SIGNAL

'Join and group by it, but never display it' is the definition of a projection policy. No other control separates use from display.

What to carry into the next module

- **Masking applies wherever the column is referenced.** JOIN, WHERE, ORDER BY, GROUP BY. Deliberately.
- **Masking is values, row access is rows, projection is presence in the output.** Three different questions.
- **Order:** row access filters, then projection rejects, then masking rewrites.
- **Tag-based masking** is the scaling answer. Attach the policy to a tag; classification applies the tag.
- **Classification samples 10,000 rows and excludes views.** It assists judgement rather than replacing it.

> ON THE EXAM
>
> Next: credits, resource monitors and the usage views. The last module of Domain 2 before its drill.